Università del Salento
Facoltà di Ingegneria
Corso di Laurea in Ingegneria dell'Informazione

# ChillChain — Piattaforma IoT per il trasporto refrigerato condiviso

*Matching domanda-offerta e rilevazione anomalie con LSTM Autoencoder*

Relatore
Prof. Luigi Patrono

Tutor
Ing. Teodoro Montanaro

Studente
Antonio Bello
Mat. 20093232

Anno Accademico 2025–2026

# Indice

**1 Introduzione** **3**

**2 Analisi dei Requisiti** **5**
2.1 Requisiti Funzionali . . . . . 5
2.1.1 Gestione degli utenti e autenticazione . . . . . 5
2.1.2 Gestione della flotta . . . . . 5
2.1.3 Creazione e gestione delle tratte . . . . . 6
2.1.4 Ricerca e prenotazione delle spedizioni . . . . . 6
2.1.5 Monitoraggio telemetrico in tempo reale . . . . . 6
2.1.6 Rilevazione delle anomalie . . . . . 7
2.1.7 Gestione delle notifiche . . . . . 7
2.2 Requisiti Non Funzionali . . . . . 8
2.2.1 Usabilità . . . . . 8
2.2.2 Sicurezza . . . . . 8
2.2.3 Modularità e manutenibilità . . . . . 8
2.2.4 Portabilità e riproducibilità del deployment . . . . . 9
2.2.5 Scalabilità del monitoraggio . . . . . 9
2.2.6 Affidabilità della comunicazione . . . . . 9

**3 Stato dell'Arte** **10**
3.1 Flock Freight – FlockDirect® Shared Truckload . . . . . 10
3.2 Quicargo – Piattaforma collaborativa con pricing dinamico . . 11
3.3 Gillespie et al. – Rilevazione anomalie IoT nel trasporto refrigerato . . . . . 12
3.4 Tran et al. – LSTM Autoencoder per anomaly detection nella supply chain . . . . . 13
3.5 Sintesi e posizionamento della soluzione proposta . . . . . 13

**4 Tecnologie Utilizzate** **15**
4.1 Spring Boot . . . . . 15
4.2 React e TypeScript . . . . . 16

4.3 MQTT e Mosquitto . . . . . 16
4.4 MongoDB . . . . . 17
4.5 Python, FastAPI e PyTorch . . . . . 18
4.6 Google Maps Platform . . . . . 18
4.7 Docker e Docker Compose . . . . . 19

**5 Soluzione Proposta 20**
5.1 Architettura del sistema . . . . . 20
5.2 Gestione degli utenti e autenticazione . . . . . 21
5.3 Marketplace: ricerca e prenotazione delle tratte . . . . . 22
5.3.1 Fase 1 — Ricerca dei veicoli disponibili . . . . . 22
5.3.2 Fase 2 — Ricerca delle tratte condivise . . . . . 23
5.3.3 Fase 3 — Validazione geografica delle tratte condivise . 23
5.3.4 Fase 4 — Ricalcolo del percorso con waypoint . . . . . 23
5.3.5 Fase 5 — Pricing con divisore esponenziale . . . . . 24
5.3.6 Selezione e conferma della tratta . . . . . 24
5.4 Generazione dei dati telemetrici . . . . . 24
5.5 Pipeline di anomaly detection . . . . . 25
5.5.1 Addestramento del modello . . . . . 25
5.5.2 Inferenza in tempo reale . . . . . 26
5.5.3 Smoothing con contatori consecutivi . . . . . 26
5.6 Gestione delle notifiche . . . . . 27
5.7 Dashboard telemetrica . . . . . 27
5.8 Deployment . . . . . 28

**6 Validazione Funzionale 29**
6.1 Ambiente di test . . . . . 29
6.2 Scenario di validazione . . . . . 30
6.2.1 Fase 1 — Configurazione della flotta (Amministratore) 30
6.2.2 Fase 2 — Prima spedizione: creazione della tratta (Cliente A) . . . . . 31
6.2.3 Fase 3 — Seconda spedizione: condivisione della tratta (Cliente B) . . . . . 31
6.2.4 Fase 4 — Gestione della tratta e avvio simulazione (Amministratore) . . . . . 32
6.2.5 Fase 5 — Rilevazione anomalia e notifica (Tecnico) . . 33
6.3 Esito della validazione . . . . . 34

**7 Conclusioni e Sviluppi Futuri 39**
7.1 Conclusioni . . . . . 39
7.2 Sviluppi futuri . . . . . 40

# Capitolo 1

# Introduzione

La gestione della catena del freddo rappresenta una delle sfide più critiche nel settore della logistica moderna. Il trasporto di merci deperibili richiede il mantenimento rigoroso di condizioni ambientali controllate lungo l'intera filiera distributiva, dalla produzione alla consegna finale. Anche una breve interruzione della temperatura può compromettere irrimediabilmente la qualità e la sicurezza dei prodotti trasportati, con conseguenze economiche rilevanti e potenziali rischi per la salute pubblica. A rendere il problema ancora più complesso contribuisce la frammentazione del mercato delle spedizioni refrigerate: da un lato, le aziende di trasporto gestiscono flotte di veicoli che spesso viaggiano con capacità parzialmente inutilizzata; dall'altro, i clienti che necessitano di spedire piccoli quantitativi si trovano costretti a sostenere costi elevati per noleggiare un intero veicolo refrigerato o a ricorrere a soluzioni LTL (*Less Than Truckload*) tradizionali, che comportano molteplici passaggi in magazzino, tempi di consegna più lunghi e un rischio maggiore di rottura della catena del freddo.

Le soluzioni attualmente disponibili sul mercato affrontano questi problemi in modo parziale e disgiunto. Piattaforme come Flock Freight hanno introdotto il concetto di *Shared Truckload* (STL), sfruttando algoritmi di intelligenza artificiale per consolidare spedizioni di clienti diversi su uno stesso veicolo e ottimizzare le rotte in maniera automatica. Tuttavia, queste piattaforme adottano un approccio flessibile in cui l'algoritmo può combinare spedizioni con percorsi anche molto diversi, e sono progettate per il trasporto merci generico senza integrare funzionalità di monitoraggio della catena del freddo né strumenti di rilevazione anomalie per spedizioni refrigerate. Sul versante del monitoraggio, il panorama delle soluzioni IoT per la *cold chain* è maturo e in forte espansione. Ciò nonostante, queste soluzioni operano tipicamente come sistemi B2B verticali, scollegati dalla fase commerciale di prenotazione e pricing, e si limitano per lo più a un monitoraggio basato

su soglie statiche che, pur essendo semplice da implementare, non è in grado di rilevare pattern anomali complessi o degradi graduali delle prestazioni dell'impianto di refrigerazione.

Il presente lavoro propone **ChillChain**, una piattaforma IoT che integra in un'unica soluzione il matching tra domanda e offerta di spedizioni refrigerate e il monitoraggio intelligente della catena del freddo con rilevazione automatica delle anomalie. L'idea alla base del progetto è semplice: consentire ai clienti di visualizzare le tratte già programmate dai trasportatori, valutarne la compatibilità con le proprie esigenze di spedizione e aggregarsi a un viaggio esistente. La rotta del veicolo viene ricalcolata dinamicamente per includere le nuove fermate, mentre un sistema di pricing basato sull'occupazione del mezzo applica un divisore esponenziale che rende il costo per singolo cliente progressivamente più conveniente all'aumentare delle spedizioni condivise, incentivando così un utilizzo più efficiente della capacità di trasporto.

Accanto alla componente di marketplace, la piattaforma integra un sistema di monitoraggio in tempo reale dei parametri operativi dei veicoli refrigerati. I dati telemetrici vengono trasmessi via MQTT e analizzati da un servizio dedicato di rilevazione anomalie. Quest'ultimo impiega un LSTM Autoencoder, addestrato sul comportamento nominale dell'impianto, per calcolare l'errore di ricostruzione di ciascuna finestra temporale di osservazione: quando tale errore supera una soglia appresa in fase di training, il sistema genera una notifica che viene inoltrata al tecnico responsabile attraverso la dashboard della piattaforma.

Dal punto di vista architetturale, ChillChain adotta un'architettura a microservizi composta da un backend Spring Boot che gestisce la logica applicativa e le API REST, un frontend React con TypeScript per l'interfaccia utente, un broker MQTT Mosquitto per la comunicazione in tempo reale, due servizi Python dedicati rispettivamente allo streaming dei dati telemetrici e alla rilevazione delle anomalie, e un database MongoDB per la persistenza. L'intera infrastruttura è orchestrata tramite Docker Compose, garantendo riproducibilità e semplicità di deployment. L'obiettivo del progetto è dimostrare la fattibilità di un approccio integrato che unisca la dimensione commerciale della logistica collaborativa con quella tecnica del monitoraggio predittivo, ponendo particolare attenzione all'usabilità dell'interfaccia e alla chiarezza con cui le informazioni vengono presentate ai diversi attori del sistema: clienti, amministratori e tecnici.

# Capitolo 2

# Analisi dei Requisiti

In questo capitolo vengono descritti i requisiti funzionali e non funzionali che il sistema ChillChain è chiamato a soddisfare. L'analisi è stata condotta considerando i tre profili utente previsti dalla piattaforma — amministratore, cliente e tecnico — e le esigenze operative che ciascuno di essi esprime nei confronti del sistema.

## 2.1 Requisiti Funzionali

I requisiti funzionali definiscono le capacità operative che la piattaforma deve offrire ai propri utenti e descrivono il comportamento atteso del sistema in risposta a specifiche azioni o eventi.

### 2.1.1 Gestione degli utenti e autenticazione

Il sistema deve consentire l'autenticazione mediante credenziali protette. A ogni utente viene assegnato un ruolo tra amministratore, cliente e tecnico, e l'accesso alle funzionalità della piattaforma deve essere regolato in base a tale ruolo. L'amministratore ha il compito di gestire la flotta e le tratte; il cliente può ricercare e prenotare spedizioni; il tecnico è responsabile del monitoraggio dello stato operativo dei veicoli refrigerati. Il meccanismo di autenticazione deve essere basato su token JWT (*JSON Web Token*) con sessioni stateless, in modo da garantire la sicurezza delle comunicazioni senza richiedere la gestione dello stato lato server.

### 2.1.2 Gestione della flotta

L'amministratore deve poter registrare nuovi veicoli nella piattaforma, specificando per ciascuno le dimensioni del vano di carico (larghezza, altezza,

lunghezza), la portata massima in peso, il costo per chilometro e l'indicazione sulla presenza o meno dell'impianto di refrigerazione. Il sistema deve consentire la visualizzazione dell'intera flotta e la rimozione dei veicoli non più operativi. Queste informazioni costituiscono la base su cui vengono calcolate la capacità residua delle tratte e la compatibilità con le richieste di spedizione.

### 2.1.3 Creazione e gestione delle tratte

L'amministratore deve poter creare nuove tratte specificando il veicolo assegnato, l'indirizzo di partenza, l'indirizzo di arrivo e la data prevista di arrivo. Il sistema deve calcolare automaticamente il percorso ottimale, la distanza in chilometri e la durata stimata del viaggio sfruttando le API di Google Maps per il geocoding degli indirizzi e il calcolo delle rotte. La polyline del percorso deve essere memorizzata per consentirne la visualizzazione su mappa nell'interfaccia utente. L'amministratore deve inoltre poter visualizzare l'elenco delle tratte attive, consultare le spedizioni associate a ciascuna tratta e procedere all'eliminazione delle tratte non più necessarie.

### 2.1.4 Ricerca e prenotazione delle spedizioni

Il cliente deve poter ricercare le tratte disponibili inserendo le caratteristiche del proprio pacco — dimensioni, peso, necessità di refrigerazione — e la data entro cui la consegna deve essere completata. Il sistema deve restituire sia le tratte ancora libere sia quelle già occupate da altre spedizioni, purché dispongano di capacità residua sufficiente ad accogliere il nuovo pacco. Per le tratte condivise, il prezzo deve essere ricalcolato applicando un meccanismo di divisione basato sul numero di spedizioni già presenti, in modo da incentivare la condivisione del veicolo. Una volta selezionata una tratta, il cliente specifica gli indirizzi di ritiro e consegna della propria spedizione: il sistema ricalcola dinamicamente il percorso del veicolo introducendo le nuove fermate come waypoint intermedi e aggiorna la polyline, la distanza complessiva e la durata stimata. La capacità residua del veicolo in termini di dimensioni e peso viene decrementata di conseguenza.

### 2.1.5 Monitoraggio telemetrico in tempo reale

Il sistema deve acquisire i dati telemetrici provenienti dai sensori installati a bordo dei veicoli refrigerati. Tali dati comprendono quindici parametri operativi: temperatura ambiente, temperatura impostata, temperatura

misurata in cabina, temperatura di saturazione dell'evaporatore e del condensatore, pressione di aspirazione e di mandata, frequenza del compressore, surriscaldamento, potenza assorbita dal compressore, potenza frigorifera dell'evaporatore, coefficiente di prestazione (COP) e tre indicatori binari relativi allo stato della porta, del ciclo di sbrinamento e della valvola di espansione. La trasmissione dei dati deve avvenire tramite il protocollo MQTT con una frequenza di campionamento di un messaggio al minuto, coerente con le dinamiche operative reali degli impianti di refrigerazione. Il tecnico deve poter visualizzare i dati telemetrici attraverso una dashboard che presenti l'andamento temporale dei parametri mediante grafici in tempo reale, con la possibilità di filtrare per veicolo e per tipologia di parametro.

### 2.1.6 Rilevazione delle anomalie

Un servizio dedicato deve analizzare in maniera continuativa i flussi telemetrici provenienti dai veicoli e identificare automaticamente condizioni operative anomale. Il sistema deve impiegare un modello LSTM Autoencoder addestrato sul comportamento nominale dell'impianto di refrigerazione: per ciascuna finestra temporale di osservazione, il modello calcola l'errore di ricostruzione e lo confronta con una soglia appresa in fase di training. Per evitare falsi positivi dovuti a fluttuazioni momentanee, il sistema deve implementare un meccanismo di smoothing basato su contatori consecutivi, in modo che un'anomalia venga confermata solo dopo un numero sufficiente di rilevazioni anomale consecutive e, analogamente, venga dichiarata rientrata solo dopo un adeguato numero di rilevazioni normali. Quando un'anomalia viene confermata, il sistema deve generare una notifica persistente, associata al veicolo interessato, e renderla disponibile al tecnico responsabile attraverso la dashboard.

### 2.1.7 Gestione delle notifiche

Il tecnico deve poter consultare l'elenco delle notifiche ricevute, ordinate cronologicamente, ciascuna contenente il nome del veicolo, il timestamp dell'evento, l'errore di ricostruzione e un messaggio descrittivo dell'anomalia rilevata. Il sistema deve distinguere tra notifiche lette e non lette e consentire al tecnico di contrassegnare come letta una notifica dopo averla presa in carico. Le notifiche devono essere accessibili esclusivamente dagli utenti con ruolo tecnico.

## 2.2 Requisiti Non Funzionali

I requisiti non funzionali specificano le qualità architetturali e operative che il sistema deve garantire trasversalmente rispetto alle singole funzionalità.

### 2.2.1 Usabilità

L'interfaccia utente deve essere progettata con particolare attenzione alla chiarezza e all'immediatezza d'uso. Le informazioni devono essere presentate in modo differenziato in base al ruolo dell'utente: l'amministratore necessita di una visione d'insieme sulla flotta e sulle tratte; il cliente richiede un'esperienza di ricerca e prenotazione semplice e guidata; il tecnico ha bisogno di una dashboard di monitoraggio che evidenzi rapidamente le situazioni critiche. La visualizzazione delle tratte su mappa, i grafici telemetrici con codifica cromatica degli stati di guasto e le notifiche con badge numerici contribuiscono a rendere l'interfaccia informativa senza risultare sovraccarica.

### 2.2.2 Sicurezza

Il sistema deve proteggere le comunicazioni tra client e server mediante autenticazione basata su token JWT. Le password degli utenti devono essere memorizzate in forma cifrata utilizzando l'algoritmo BCrypt. L'accesso alle API deve essere regolato da un sistema di autorizzazione basato sui ruoli, in modo che ciascun endpoint sia raggiungibile esclusivamente dagli utenti che dispongono dei permessi necessari. Le sessioni devono essere gestite in modalità stateless per evitare vulnerabilità legate alla gestione dello stato lato server.

### 2.2.3 Modularità e manutenibilità

L'architettura del sistema deve seguire il paradigma a microservizi, con una separazione netta tra le componenti applicative: il backend per la logica di business, il frontend per l'interfaccia utente, il broker MQTT per la comunicazione asincrona, i servizi Python per lo streaming telemetrico e la rilevazione delle anomalie, e il database per la persistenza. Ogni componente deve poter essere sviluppato, testato e aggiornato indipendentemente dagli altri, favorendo la manutenibilità nel tempo e la possibilità di sostituire o estendere singoli moduli senza impattare sul resto del sistema.

### 2.2.4 Portabilità e riproducibilità del deployment

L'intera infrastruttura deve essere containerizzata tramite Docker e orchestrata mediante Docker Compose, in modo da garantire che il sistema possa essere avviato in maniera identica su qualsiasi macchina che disponga del runtime Docker, indipendentemente dal sistema operativo e dalla configurazione dell'ambiente ospitante. Il file di orchestrazione deve definire tutti i servizi, le reti interne e le dipendenze tra i container, rendendo il deployment un'operazione deterministica e ripetibile.

### 2.2.5 Scalabilità del monitoraggio

Il servizio di rilevazione anomalie deve essere in grado di gestire simultaneamente i flussi telemetrici provenienti da più veicoli, mantenendo per ciascuno un buffer indipendente e uno stato interno separato. L'aggiunta di un nuovo veicolo monitorato non deve richiedere interventi manuali di configurazione: il sistema deve creare automaticamente un'istanza del detector alla ricezione del primo messaggio telemetrico da un veicolo precedentemente sconosciuto e rilasciare le risorse associate al completamento dello stream.

### 2.2.6 Affidabilità della comunicazione

La comunicazione tra i servizi di telemetria e il backend deve avvenire tramite il protocollo MQTT, che garantisce un disaccoppiamento tra produttori e consumatori dei messaggi e supporta la riconnessione automatica in caso di interruzioni temporanee della connettività. Il sistema deve gestire in maniera controllata la disconnessione e la riconnessione dei client MQTT, assicurando che nessun dato telemetrico venga perso durante transitori di rete e che lo stato interno dei detector venga correttamente reinizializzato quando necessario.

# Capitolo 3

# Stato dell'Arte

In questo capitolo vengono analizzate le principali soluzioni esistenti nel panorama della logistica collaborativa, del monitoraggio IoT della catena del freddo e della rilevazione di anomalie su serie temporali. Per ciascuna soluzione viene fornita una descrizione sintetica, seguita da un'analisi critica che ne evidenzia i limiti rispetto alla proposta presentata in questo lavoro.

## 3.1 Flock Freight – FlockDirect® Shared Truckload

Flock Freight è un'azienda statunitense che ha introdotto il concetto di *Shared Truckload* (STL), una modalità di spedizione che consente di consolidare carichi di clienti diversi su un unico camion mediante una tecnologia brevettata basata su intelligenza artificiale. L'algoritmo di Flock Freight analizza posizioni geografiche, calendari di spedizione e dimensioni dei carichi per individuare, tra miliardi di combinazioni possibili, gli abbinamenti più efficienti, instradando il veicolo su percorsi diretti e privi di soste in magazzini intermedi [2]. A differenza del tradizionale LTL (*Less Than Truckload*), in cui la merce viene scaricata e ricaricata più volte lungo il percorso, nello STL il carico non lascia mai il veicolo fino alla consegna, riducendo drasticamente il rischio di danneggiamento e i tempi di transito. Flock Freight dichiara consegne puntuali nel 97% dei casi e un tasso di danneggiamento 5,4 volte inferiore rispetto all'LTL, con una riduzione delle emissioni di $CO_2$ fino al 40%.

Nonostante la solidità della soluzione, il modello di Flock Freight presenta caratteristiche differenti rispetto all'approccio proposto in ChillChain. Dal punto di vista dell'ottimizzazione, Flock adotta un approccio flessibile basato su AI che può combinare spedizioni con percorsi anche molto diversi

tra loro, massimizzando il numero di abbinamenti possibili sull'intera rete. ChillChain segue invece un approccio spazialmente più rigido: le spedizioni condivise devono trovarsi lungo il percorso già programmato del veicolo, con una tolleranza massima di deviazione sia in termini di distanza dalla rotta sia di incremento della durata del viaggio. Questo vincolo riduce il numero di combinazioni possibili, ma quando il matching avviene il risparmio è potenzialmente più elevato, poiché la deviazione introdotta è minima e il divisore esponenziale $2^n$ scala in modo aggressivo con il numero di spedizioni a bordo. Si tratta di due approcci alternativi, ciascuno con i propri compromessi tra flessibilità e ottimizzazione del costo. Inoltre, la piattaforma Flock Freight è progettata per il trasporto merci generico e non integra alcuna funzionalità di monitoraggio della catena del freddo: non offre strumenti di telemetria né di rilevazione anomalie per spedizioni refrigerate.

## 3.2 Quicargo – Piattaforma collaborativa con pricing dinamico

Quicargo è una piattaforma logistica digitale fondata nei Paesi Bassi che opera come intermediario tra mittenti e trasportatori nel segmento LTL. La piattaforma consente ai clienti di ottenere preventivi istantanei e prenotare spedizioni di pallet verso 29 Paesi europei attraverso un'interfaccia digitale, senza contratti a lungo termine né costi nascosti [4]. Un aspetto particolarmente rilevante dal punto di vista scientifico è il lavoro di Atasoy, Schulte e Steenkamp (2020), che hanno studiato il caso Quicargo formulando un problema collaborativo di *pickup and delivery* con finestre temporali (PDPTW), in cui il pricing pagato dalla piattaforma ai trasportatori viene modulato dinamicamente per bilanciare gli interessi di tutti gli attori coinvolti [1]. Il modello proposto dimostra che la collaborazione basata su piattaforma consente di ridurre significativamente i costi complessivi rispetto a un approccio non collaborativo, a condizione che vengano garantiti vincoli di razionalità individuale per i trasportatori.

Il contributo di Quicargo e della ricerca associata è rilevante per la formalizzazione del problema di *routing* collaborativo con incentivi economici. Tuttavia, la piattaforma presenta diversi limiti rispetto alla soluzione proposta nel presente lavoro. La logica di routing e abbinamento resta interamente lato piattaforma: il cliente non visualizza le tratte esistenti e non ha la possibilità di selezionare un viaggio specifico a cui aggregarsi. Il pricing dinamico, per quanto sofisticato nella formulazione accademica, non viene reso trasparente all'utente finale, che non dispone di informazioni sulla composizione del

costo né sull'effetto della condivisione del veicolo sul prezzo. Inoltre, analogamente a Flock Freight, Quicargo opera esclusivamente nel trasporto merci generico e non offre alcuna funzionalità legata al monitoraggio della catena del freddo, alla telemetria dei veicoli refrigerati o alla rilevazione di anomalie operative.

## 3.3 Gillespie et al. – Rilevazione anomalie IoT nel trasporto refrigerato

Gillespie et al. (2023) propongono un sistema IoT per la rilevazione in tempo reale di anomalie durante il trasporto refrigerato di prodotti alimentari deperibili, pubblicato sulla rivista *Sustainability* [3]. Il sistema si basa sul data logger cellulare Digital Matter Eagle, collegato a due sonde di temperatura posizionate all'interno del vano refrigerato del veicolo. I dati vengono trasmessi via rete LTE-M a una piattaforma cloud, dove un algoritmo di alerting basato su soglie di temperatura genera notifiche destinate al personale di magazzino e ai conducenti. Il sistema include la capacità di rilevare se il veicolo è in movimento, in modo da attivare gli allarmi esclusivamente durante le fasi di trasporto ed evitare falsi positivi quando il mezzo è parcheggiato e l'impianto di refrigerazione è spento.

Il lavoro di Gillespie et al. rappresenta un riferimento importante per il monitoraggio IoT applicato alla catena del freddo, in quanto dimostra la fattibilità di un sistema di alerting in tempo reale a basso costo basato su hardware commerciale. Tuttavia, la soluzione presenta limitazioni rilevanti sotto diversi profili. Dal punto di vista della rilevazione delle anomalie, il sistema si basa esclusivamente su soglie statiche di temperatura: viene generato un allarme quando il valore misurato supera o scende sotto una soglia predefinita. Questo approccio, pur essendo semplice e robusto, non è in grado di identificare pattern anomali complessi, come degradi progressivi delle prestazioni dell'impianto di refrigerazione o correlazioni tra parametri multipli (pressioni, frequenza del compressore, coefficiente di prestazione) che potrebbero indicare un guasto incipiente prima che si manifesti come una deviazione di temperatura. Inoltre, il sistema monitora esclusivamente due sonde di temperatura e non acquisisce i parametri operativi dell'impianto frigorifero, limitando la capacità diagnostica. Infine, la soluzione è confinata al solo monitoraggio: non è integrata con alcuna piattaforma di gestione delle spedizioni né con un sistema di matching tra domanda e offerta.

## 3.4 Tran et al. – LSTM Autoencoder per anomaly detection nella supply chain

Tran, Heuchenne e Thomassey (2021) propongono un approccio basato sulla combinazione di LSTM Autoencoder e *Isolation Forest* per la rilevazione di anomalie su serie temporali multivariate nel contesto della gestione della *supply chain*, con applicazione specifica al settore della moda [6]. Il modello sfrutta la capacità degli LSTM Autoencoder di apprendere le dipendenze temporali a lungo termine presenti nei dati di vendita e di generare una rappresentazione compressa (*latent space*) del comportamento normale del sistema. Le anomalie vengono identificate calcolando l'errore di ricostruzione tra l'input originale e l'output del decoder: i campioni che presentano un errore superiore a una soglia appresa durante l'addestramento vengono classificati come anomali. I risultati sperimentali mostrano che l'approccio basato su LSTM Autoencoder raggiunge prestazioni superiori rispetto ai metodi tradizionali nella rilevazione di anomalie su dati sequenziali.

Il lavoro di Tran et al. è particolarmente rilevante per il presente progetto poiché fornisce la base metodologica per l'impiego degli LSTM Autoencoder nella rilevazione di anomalie su serie temporali multivariate. Tuttavia, l'applicazione è limitata al dominio delle vendite e della *demand forecasting*, un contesto significativamente diverso da quello del monitoraggio di impianti di refrigerazione. Il modello opera in modalità *offline* su dataset storici e non affronta le problematiche legate all'elaborazione in tempo reale di flussi continui di dati telemetrici, quali la gestione di buffer a finestra scorrevole, la necessità di meccanismi di smoothing per evitare falsi positivi transitori e il supporto al monitoraggio simultaneo di più sorgenti dati indipendenti. Inoltre, il sistema è un componente puramente analitico, privo di integrazione con piattaforme operative di notifica, dashboard di visualizzazione o sistemi di gestione delle spedizioni.

## 3.5 Sintesi e posizionamento della soluzione proposta

Dall'analisi dello stato dell'arte emerge che le soluzioni esistenti affrontano in modo separato i tre ambiti coinvolti nel presente lavoro: il matching tra domanda e offerta di spedizioni, il monitoraggio IoT della catena del freddo e la rilevazione di anomalie basata su *deep learning*. Le piattaforme di trasporto collaborativo come Flock Freight e Quicargo offrono meccanismi di consolidamento dei carichi ma non consentono al cliente di confrontare e

scegliere tra più alternative di trasporto, non implementano un sistema di pricing basato sull'occupazione del veicolo e non integrano funzionalità di monitoraggio per spedizioni refrigerate. Il sistema IoT di Gillespie et al. dimostra l'efficacia del monitoraggio in tempo reale ma si limita a due sonde di temperatura con soglie statiche, senza sfruttare le potenzialità del *machine learning* per la rilevazione di anomalie multivariate. Il lavoro di Tran et al. valida l'approccio LSTM Autoencoder per l'anomaly detection su serie temporali ma in un contesto applicativo differente e senza affrontare le sfide dell'elaborazione *online* e dell'integrazione in un sistema operativo completo.

ChillChain si propone di colmare questo gap attraverso una piattaforma integrata che combina un marketplace con selezione trasparente delle tratte da parte del cliente e pricing dinamico basato sull'occupazione del veicolo, un sistema di monitoraggio in tempo reale di quindici parametri operativi dell'impianto frigorifero trasmessi via MQTT, e un servizio di rilevazione anomalie basato su LSTM Autoencoder con meccanismo di smoothing, operante in modalità streaming su flussi telemetrici multipli e integrato con un sistema di notifiche destinato al personale tecnico.

# Capitolo 4

# Tecnologie Utilizzate

In questo capitolo vengono descritte le principali tecnologie impiegate nella realizzazione di ChillChain, evidenziando per ciascuna gli aspetti specifici che hanno contribuito alla soluzione adottata. La scelta dello stack tecnologico è stata guidata dalla necessità di integrare in un'unica piattaforma componenti eterogenee — un backend per la logica di business, un frontend interattivo, servizi di streaming e analisi dati, un broker per la comunicazione asincrona e un database orientato ai documenti — garantendo al contempo semplicità di deployment e riproducibilità dell'ambiente di esecuzione.

## 4.1 Spring Boot

Spring Boot è il framework Java utilizzato per la realizzazione del backend della piattaforma. La scelta è ricaduta su questo framework per la sua capacità di semplificare la configurazione di applicazioni enterprise, eliminando gran parte del boilerplate tipico dello sviluppo con Spring tradizionale grazie al principio della *convention over configuration*.

Nel contesto di ChillChain, Spring Boot è stato impiegato per esporre le API REST che gestiscono l'intero ciclo di vita delle entità di dominio: veicoli, tratte, spedizioni, notifiche e utenti. Il modulo Spring Security è stato configurato per implementare l'autenticazione basata su token JWT con sessioni stateless, definendo una catena di filtri che intercetta ogni richiesta HTTP e ne verifica l'autorizzazione in base al ruolo dell'utente (amministratore, cliente, tecnico). L'integrazione con MongoDB è stata realizzata tramite Spring Data MongoDB, che consente di definire repository dichiarativi con query derivate dal nome dei metodi e pipeline di aggregazione personalizzate — queste ultime utilizzate, ad esempio, per calcolare il prezzo delle tratte condivise eseguendo *join* tra collezioni e applicando la logica di

divisione direttamente a livello di database. Un aspetto particolarmente rilevante è l'integrazione con il protocollo MQTT tramite Spring Integration: il backend sottoscrive i topic di telemetria e anomalie pubblicati dai servizi Python, riceve i messaggi attraverso channel adapter dedicati e li instrada a service activator che si occupano della persistenza dei dati e della generazione delle notifiche.

## 4.2 React e TypeScript

Il frontend della piattaforma è stato realizzato con React, una libreria JavaScript per la costruzione di interfacce utente basate su componenti, in combinazione con TypeScript, un superset tipizzato di JavaScript che aggiunge il controllo statico dei tipi.

L'impiego di React ha permesso di strutturare l'interfaccia come un insieme di componenti riutilizzabili, ciascuno responsabile di una porzione specifica della vista: la lista dei veicoli, il form di invio spedizione, la mappa delle tratte, la dashboard telemetrica e il pannello notifiche. La gestione dello stato applicativo si basa sugli hook nativi di React (`useState`, `useEffect`) e su `localStorage` per la persistenza delle informazioni di sessione attraverso la navigazione. TypeScript ha contribuito a ridurre gli errori a runtime fornendo un sistema di tipi che rende espliciti i contratti tra componenti, particolarmente utile nella gestione dei DTO provenienti dal backend. La visualizzazione dei dati telemetrici in tempo reale è stata realizzata con la libreria Recharts, che consente di costruire grafici ad area con aggiornamento dinamico e codifica cromatica in base allo stato di guasto. La visualizzazione cartografica delle tratte si appoggia a React-Leaflet, impiegato per il rendering delle polyline dei percorsi su mappa interattiva. L'intero progetto frontend utilizza Vite come build tool, scelto per la velocità del server di sviluppo e il supporto nativo al *hot module replacement*, e Tailwind CSS come framework di utility per lo stile, che ha permesso di ottenere un'interfaccia coerente e responsive senza scrivere fogli di stile personalizzati.

## 4.3 MQTT e Mosquitto

MQTT (*Message Queuing Telemetry Transport*) è il protocollo di comunicazione adottato per lo scambio di dati telemetrici tra i servizi della piattaforma. Si tratta di un protocollo di messaggistica leggero basato sul pattern *publish/subscribe*, progettato specificamente per contesti IoT caratterizzati da banda limitata e risorse computazionali ridotte.

In ChillChain, MQTT costituisce il tessuto connettivo tra tre attori: il servizio di streaming (publisher), il servizio di anomaly detection (subscriber e publisher) e il backend Spring Boot (subscriber). Il broker Mosquitto, un'implementazione open source del protocollo MQTT, gestisce l'instradamento dei messaggi attraverso una struttura gerarchica di topic nella forma `fridge/{vehicleName}/telemetry` per i dati sensoristici e `fridge/{vehicleName}/anomalies` per i risultati dell'analisi. L'uso del carattere jolly `+` nelle sottoscrizioni consente a ciascun subscriber di ricevere i messaggi relativi a tutti i veicoli senza dover conoscere a priori i nomi delle sorgenti. Questa architettura disaccoppia completamente i produttori dai consumatori dei dati: il servizio di streaming pubblica i messaggi senza sapere chi li riceverà, e il servizio di anomaly detection può essere avviato, fermato o sostituito senza impattare sugli altri componenti. La riconnessione automatica, configurata sia lato Python (tramite la libreria Paho MQTT) sia lato Spring Boot (tramite le opzioni del client factory), garantisce la resilienza della comunicazione in caso di interruzioni temporanee del broker.

## 4.4 MongoDB

MongoDB è il database NoSQL orientato ai documenti impiegato per la persistenza di tutti i dati della piattaforma. A differenza dei database relazionali, MongoDB memorizza i dati in documenti JSON-like (BSON) organizzati in collezioni, senza imporre uno schema rigido.

Nel contesto di ChillChain, questa flessibilità si è rivelata particolarmente vantaggiosa per la gestione dei dati telemetrici, la cui struttura può variare in funzione del tipo di sensore, e per le entità di dominio come tratte e spedizioni, che includono campi con struttura annidata (coordinate geografiche, polyline). Il sistema utilizza sei collezioni principali: `vehicle`, `trip`, `shipment`, `notification`, `telemetry` e `user`. L'interazione avviene tramite Spring Data MongoDB, che mappa automaticamente le classi Java alle collezioni corrispondenti. Un aspetto specifico del progetto è l'uso delle pipeline di aggregazione di MongoDB per implementare la logica di ricerca delle tratte condivise: una singola query esegue *lookup* tra le collezioni `trip`, `vehicle` e `shipment`, filtra per capacità residua, tipologia di refrigerazione e data di arrivo, e calcola il prezzo scontato dividendo il costo minimo per il numero di spedizioni, il tutto a livello di database senza necessità di elaborazione applicativa. Gli indici geospaziali sulle coordinate di partenza e arrivo delle tratte e delle spedizioni consentono inoltre di eseguire query di prossimità per future estensioni del sistema.

## 4.5 Python, FastAPI e PyTorch

Python è il linguaggio utilizzato per i due microservizi dedicati rispettivamente allo streaming dei dati telemetrici e alla rilevazione delle anomalie. La scelta è motivata dalla ricchezza dell'ecosistema Python per il *data processing* e il *machine learning*, che ha consentito di implementare entrambi i servizi con un numero ridotto di dipendenze.

Il servizio di streaming (`streamer_main.py`) è un'applicazione FastAPI che legge i file CSV contenenti i dati simulati di telemetria e li pubblica sul broker MQTT con una cadenza configurabile, simulando il comportamento di sensori reali installati a bordo dei veicoli. I dataset sono generati da uno script di simulazione termodinamica (`stream_fridge_dataset_generator.py`), derivato dal dataset Kaggle *Simulated Refrigerator Fault Diagnosis* [5], che modella il comportamento fisico di un impianto frigorifero a compressione di vapore includendo scambi termici con l'ambiente, cicli di sbrinamento, aperture della porta e tredici tipologie di guasto iniettabili con parametri stocastici. Questo approccio consente di generare scenari realistici di funzionamento normale e anomalo con pieno controllo sulle condizioni sperimentali.

Il servizio di anomaly detection (`anomaly_main.py`) è anch'esso un'applicazione FastAPI che sottoscrive il topic telemetria via MQTT, analizza i dati in ingresso e pubblica i risultati sul topic anomalie. Il cuore del servizio è un modello LSTM Autoencoder implementato in PyTorch: l'encoder, una rete LSTM, comprime la sequenza temporale di ingresso in una rappresentazione latente a dimensionalità ridotta; il decoder, un'altra rete LSTM, ricostruisce la sequenza originale a partire da tale rappresentazione. L'errore di ricostruzione, calcolato come errore quadratico medio (MSE) tra input e output, viene confrontato con una soglia appresa durante l'addestramento su dati di funzionamento normale. Lo scaler, serializzato con Pickle, normalizza le quindici feature in ingresso per garantire che parametri con scale numeriche molto diverse (temperature in gradi, pressioni in bar, potenze in watt) contribuiscano in modo equilibrato all'errore di ricostruzione. La libreria Pandas è impiegata per la lettura dei CSV e la preparazione dei DataFrame, mentre NumPy gestisce le operazioni numeriche sui buffer a finestra scorrevole.

## 4.6 Google Maps Platform

Le API di Google Maps Platform sono utilizzate per il calcolo dei percorsi e la geolocalizzazione degli indirizzi inseriti dagli utenti. In particolare, il servizio di backend impiega due API distinte: la *Geocoding API*, che converte gli indirizzi testuali in coordinate geografiche (latitudine e longitudine), e la

*Routes API*, che calcola il percorso ottimale tra due o più punti restituendo la distanza in metri, la durata stimata e la polyline codificata del tracciato.

Nel flusso operativo di ChillChain, queste API vengono invocate in due momenti: alla creazione di una nuova tratta da parte dell'amministratore, per calcolare il percorso base tra partenza e arrivo, e all'inserimento di una nuova spedizione condivisa, per ricalcolare il percorso con i waypoint intermedi corrispondenti ai punti di ritiro e consegna del nuovo pacco. La polyline restituita dall'API viene memorizzata nel database e trasmessa al frontend, dove viene decodificata e renderizzata su mappa tramite React-Leaflet. L'invocazione delle API avviene lato server tramite `RestTemplate` di Spring, evitando di esporre la chiave API al client.

## 4.7 Docker e Docker Compose

Docker è la piattaforma di containerizzazione impiegata per isolare ciascun componente dell'architettura in un ambiente di esecuzione autonomo e riproducibile. Ogni servizio della piattaforma — il backend Spring Boot, il frontend React, il broker Mosquitto, il servizio di streaming Python, il servizio di anomaly detection e il database MongoDB — viene eseguito all'interno di un container dedicato, con le proprie dipendenze e la propria configurazione.

Docker Compose orchestra l'avvio coordinato di tutti i container, definendo in un unico file YAML le immagini, le porte esposte, le variabili d'ambiente, i volumi persistenti e le reti interne. La rete Docker interna consente ai servizi di comunicarsi utilizzando i nomi dei container come hostname (ad esempio, il backend raggiunge MongoDB tramite l'hostname `mongodb` e il broker MQTT tramite `mosquitto`), eliminando la necessità di configurare indirizzi IP statici. Questa scelta garantisce che l'intero sistema possa essere avviato con un singolo comando `docker-compose up`, riproducendo un ambiente identico su qualsiasi macchina dotata del runtime Docker, indipendentemente dal sistema operativo sottostante.

# Capitolo 5

# Soluzione Proposta

In questo capitolo viene descritta in dettaglio la soluzione realizzata, partendo dall'architettura complessiva del sistema per poi approfondire il funzionamento logico di ciascun componente e i principali dettagli implementativi. La piattaforma ChillChain si compone di due macro-funzionalità integrate: un marketplace per il trasporto condiviso refrigerato e un sistema di monitoraggio in tempo reale con rilevazione automatica delle anomalie.

## 5.1 Architettura del sistema

L'architettura di ChillChain segue il paradigma a microservizi, in cui ciascun componente è responsabile di una funzione specifica e comunica con gli altri attraverso interfacce ben definite. La Figura 5.1 illustra schematicamente i servizi e i flussi di comunicazione.

*Diagramma architetturale del sistema*

Figura 5.1: Architettura a microservizi di ChillChain. Le frecce continue rappresentano comunicazioni HTTP/REST, le frecce tratteggiate i flussi MQTT.

Il sistema è composto da sei servizi containerizzati, orchestrati tramite Docker Compose:

Il **backend Spring Boot** (porta 8081) rappresenta il nucleo applicativo: espone le API REST per la gestione di veicoli, tratte, spedizioni, utenti e notifiche, implementa l'autenticazione JWT e la logica di autorizzazione basata sui ruoli, sottoscrive i topic MQTT per ricevere telemetria e risultati dell'anomaly detection, e persiste tutti i dati su MongoDB.

Il **frontend React** (porta 5173) fornisce l'interfaccia utente, differenziata per ruolo: l'amministratore gestisce la flotta e le tratte, il cliente cerca e prenota spedizioni, il tecnico monitora i veicoli e riceve le notifiche di anomalia.

Il **broker MQTT Mosquitto** (porta 1883) gestisce la comunicazione asincrona tra i servizi Python e il backend, instradando i messaggi attraverso topic gerarchici.

Il **servizio di streaming** (porta 8002) simula i sensori a bordo dei veicoli, leggendo dataset CSV e pubblicando i dati telemetrici sul broker MQTT con una cadenza di un messaggio al secondo.

Il **servizio di anomaly detection** (porta 8003) sottoscrive la telemetria pubblicata dallo streamer, la analizza con il modello LSTM Autoencoder e pubblica i risultati su un topic dedicato.

Infine, **MongoDB** (porta 27017) funge da database centralizzato, organizzato in sei collezioni: `user`, `vehicle`, `trip`, `shipment`, `telemetry` e `notification`.

La comunicazione tra i servizi avviene su due canali distinti: le API REST, utilizzate per le interazioni sincrone tra frontend e backend, e il protocollo MQTT, impiegato per i flussi di dati telemetrici che richiedono comunicazione asincrona e disaccoppiamento tra produttori e consumatori. Questa separazione consente di avviare, fermare o sostituire i servizi Python senza impattare sul funzionamento del backend o del frontend.

## 5.2 Gestione degli utenti e autenticazione

Il sistema prevede tre ruoli distinti, ciascuno con un perimetro di azioni e una vista dell'interfaccia dedicati:

- **ADMIN**: gestisce la flotta (aggiunta e rimozione di veicoli), visualizza e cancella tratte e spedizioni, avvia e interrompe le simulazioni di telemetria.

- **CLIENT**: ricerca tratte compatibili con le proprie esigenze di spedizione, confronta le opzioni disponibili (veicoli vuoti e tratte condivise) e prenota la spedizione.
- **TECHNICIAN**: accede alla dashboard di monitoraggio, visualizza lo storico telemetrico dei veicoli e gestisce le notifiche di anomalia.

L'autenticazione è implementata tramite token JWT (*JSON Web Token*) con sessioni stateless. All'atto del login, il backend verifica le credenziali — la password è memorizzata con hashing BCrypt — e genera un token firmato contenente l'email dell'utente e il suo ruolo. Il token viene incluso nell'header `Authorization` di ogni richiesta successiva e validato da un filtro che precede l'elaborazione della richiesta stessa. La catena di sicurezza Spring Security definisce quali endpoint sono accessibili a ciascun ruolo: ad esempio, le operazioni di gestione della flotta richiedono il ruolo `ADMIN`, mentre l'accesso alla telemetria e alle notifiche è riservato al ruolo `TECHNICIAN`. Gli endpoint di registrazione, autenticazione e ricerca tratte sono invece pubblici, così da consentire l'onboarding autonomo dei clienti.

## 5.3 Marketplace: ricerca e prenotazione delle tratte

Il cuore commerciale della piattaforma è il meccanismo che consente ai clienti di cercare tratte compatibili con le proprie esigenze e di prenotare spedizioni, eventualmente condividendo il veicolo con altri clienti. Il flusso si articola in cinque fasi.

### 5.3.1 Fase 1 — Ricerca dei veicoli disponibili

Quando un cliente inserisce i dati della propria spedizione (indirizzo di partenza e arrivo, dimensioni, peso, necessità di refrigerazione e data di arrivo desiderata), il backend esegue una prima query per individuare i veicoli della flotta che soddisfano i requisiti dimensionali e non hanno ancora alcuna tratta assegnata. Questa query è implementata come aggregation pipeline su MongoDB: il primo stadio filtra i veicoli per peso massimo, dimensioni e tipologia di refrigerazione; il secondo esegue un `$lookup` sulla collezione `trip`; il terzo seleziona solo i veicoli il cui array di tratte risulta vuoto. Per ciascun veicolo disponibile, il sistema invoca la Routes API di Google Maps per calcolare il percorso tra la partenza e l'arrivo indicati dal cliente, ottenendo distanza, durata e polyline codificata. Il prezzo viene calcolato come

prezzo = distanceKm $\times$ pricePerKm, dove `pricePerKm` è il costo chilometrico associato al veicolo.

### 5.3.2 Fase 2 — Ricerca delle tratte condivise

Parallelamente, il sistema cerca tratte già programmate che abbiano capacità residua sufficiente e data di arrivo compatibile. Anche questa ricerca è implementata come aggregation pipeline: il primo stadio esegue un `$lookup` con la collezione `vehicle` per recuperare la tipologia di refrigerazione; il secondo filtra per capacità residua, refrigerazione e data; il terzo esegue un ulteriore `$lookup` con la collezione `shipment` per calcolare il prezzo scontato come price = min(shipments.price) $\div$ 2.

### 5.3.3 Fase 3 — Validazione geografica delle tratte condivise

Le tratte condivise individuate nella fase precedente vengono ulteriormente filtrate verificando la compatibilità geografica con la spedizione richiesta. Per ciascuna tratta candidata, il sistema decodifica la polyline del percorso attuale e calcola la distanza geodetica (formula di Haversine) tra i punti di ritiro e consegna del nuovo pacco e il tracciato della tratta. Se entrambi i punti distano meno di 1,5 km dalla polyline, la tratta è considerata geograficamente compatibile. Un ulteriore controllo verifica che il punto di ritiro preceda quello di consegna lungo il percorso, impedendo deviazioni illogiche.

### 5.3.4 Fase 4 — Ricalcolo del percorso con waypoint

Per le tratte che superano la validazione geografica, il sistema ricalcola il percorso includendo i waypoint intermedi di tutti gli shipment (sia quelli già assegnati sia quello nuovo). L'ordinamento dei waypoint è affidato a un algoritmo greedy con vincoli: partendo dalla posizione corrente, seleziona il waypoint disponibile più vicino, rispettando il vincolo che il punto di ritiro di ciascuna spedizione debba essere visitato prima del corrispondente punto di consegna. Un delivery diventa "disponibile" solo dopo che il relativo pickup è stato inserito nella sequenza. Il percorso ricalcolato viene confrontato con quello originale: se la differenza di durata non supera i 30 minuti, la tratta viene inclusa tra i risultati.

### 5.3.5 Fase 5 — Pricing con divisore esponenziale

Il sistema di pricing incentiva la condivisione attraverso un divisore esponenziale. Se il veicolo ospita già $n$ spedizioni, il prezzo per il nuovo cliente viene calcolato come:

$$\text{prezzo} = \frac{\text{distanceKm}_{\text{shipment}} \times \text{pricePerKm}_{\text{vehicle}}}{2^n} \tag{1}$$

Dove $\text{distanceKm}_{\text{shipment}}$ è la distanza del tratto specifico del nuovo cliente e $n$ è il numero di spedizioni già presenti. In questo modo, la prima spedizione paga il prezzo pieno, la seconda paga metà, la terza un quarto e così via. Questo meccanismo crea un incentivo economico crescente: più il veicolo è occupato, più il trasporto diventa conveniente per i nuovi clienti, incentivando il riempimento del mezzo e riducendo i costi per tutti i partecipanti.

### 5.3.6 Selezione e conferma della tratta

Il cliente visualizza tutte le opzioni — veicoli vuoti e tratte condivise — ordinate per prezzo, e può esaminare il percorso su mappa prima di confermare. Al momento della selezione, il backend esegue un'operazione transazionale: se la tratta è nuova (veicolo senza trip assegnato), viene creata una nuova entità `Trip` con la capacità residua decrementata delle dimensioni del pacco; se la tratta è condivisa, viene aggiornata l'entità `Trip` esistente con il nuovo percorso (inclusivo dei waypoint) e la capacità residua aggiornata. In entrambi i casi, viene creata un'entità `Shipment` associata alla tratta.

## 5.4 Generazione dei dati telemetrici

La simulazione della telemetria di bordo si basa su un modello termodinamico che riproduce il comportamento fisico di un impianto frigorifero a compressione di vapore. Il generatore, derivato dal dataset Kaggle *Simulated Refrigerator Fault Diagnosis* [5] ed esteso per le esigenze del progetto, modella i seguenti fenomeni: lo scambio termico tra l'ambiente esterno e la cella frigorifera attraverso le pareti (parametrizzato dal coefficiente $UA_{\text{cab}}$), il ciclo di compressione con controllo proporzionale della velocità del compressore rispetto al setpoint di temperatura, il calcolo del COP (*Coefficient of Performance*) basato sul ciclo di Carnot con fattore di efficienza, l'accumulo e lo scioglimento del ghiaccio sull'evaporatore, le aperture casuali della porta con carichi termici variabili e i cicli periodici di sbrinamento.

Il generatore produce dataset CSV contenenti 120 campioni (120 minuti con campionamento a 60 secondi), strutturati in due fasi: i primi 60 minuti

in condizioni normali, seguiti da 60 minuti con un guasto iniettato. Sono previste 13 tipologie di guasto, tra cui: fouling del condensatore (lieve e severo), degradazione e guasto della ventola dell'evaporatore, sottocariamento e sovracariamento del refrigerante, deriva del sensore di temperatura, inefficienza del compressore, presenza di gas non condensabili e guasti compositi. Ciascun guasto è parametrizzato con un range stocastico, generando variabilità tra le simulazioni.

Il servizio di streaming legge questi CSV e pubblica ciascuna riga sul topic MQTT `fridge/{vehicleName}/telemetry` con un intervallo configurabile (default: 1 secondo), includendo un timestamp ISO e un indice di riga progressivo. Al termine del dataset, pubblica un messaggio di completamento con `stream_status: "completed"`, che segnala agli altri servizi la fine dello stream.

## 5.5 Pipeline di anomaly detection

La rilevazione delle anomalie si articola in tre fasi: l'addestramento offline del modello, l'inferenza in tempo reale e il meccanismo di smoothing per la riduzione dei falsi positivi.

### 5.5.1 Addestramento del modello

Il modello LSTM Autoencoder è stato addestrato su Google Colab con GPU Tesla T4, utilizzando un dataset composto esclusivamente da dati di funzionamento normale. L'architettura del modello è la seguente: un encoder LSTM con 15 feature in ingresso e hidden size pari a 16, uno strato lineare di compressione che riduce la rappresentazione a uno spazio latente di dimensione 8, due strati lineari che inizializzano hidden state e cell state del decoder a partire dalla rappresentazione latente, un decoder LSTM con hidden size 16 alimentato da una sequenza di zeri (la ricostruzione dipende interamente dallo stato latente) e uno strato lineare di output che proietta la dimensione nascosta nelle 15 feature originali.

L'addestramento è stato condotto per 100 epoche con batch size di 64 e finestre temporali di 30 campioni. L'ottimizzatore Adam con learning rate iniziale $10^{-3}$ è stato affiancato da uno scheduler `ReduceLROnPlateau` che dimezza il learning rate dopo 5 epoche senza miglioramento della validation loss. Il training utilizza mixed precision (`GradScaler`) per accelerare il calcolo su GPU. Un meccanismo di early stopping con pazienza di 10 epoche previene l'overfitting. Il dataset di training è stato suddiviso 80/20 in modo sequenziale (senza shuffle, per preservare la struttura temporale). La

validation loss finale è risultata pari a circa 0,00398, con un tempo di addestramento complessivo di circa 715 secondi. Le 15 feature in ingresso vengono normalizzate mediante uno `StandardScaler` di scikit-learn, serializzato con Pickle e caricato a runtime dal servizio di inferenza.

La soglia di anomalia è stata determinata calcolando l'MSE di ricostruzione su tutto il dataset di training normale e prendendo il 95° percentile della distribuzione risultante (threshold $= 0,009868$). Questo approccio garantisce che il 95% delle osservazioni normali produca un errore inferiore alla soglia, limitando il tasso di falsi positivi al 5% teorico in condizioni nominali.

### 5.5.2 Inferenza in tempo reale

Il servizio di anomaly detection sottoscrive il topic `fridge/+/telemetry` tramite il broker MQTT, ricevendo i dati di tutti i veicoli. Per ciascun veicolo viene creato dinamicamente un'istanza di `VehicleAnomalyDetector` alla ricezione del primo messaggio, garantendo il supporto simultaneo di più veicoli con buffer e contatori indipendenti.

Ad ogni messaggio ricevuto, le 15 feature vengono estratte, normalizzate con lo scaler e inserite in un buffer a finestra scorrevole di 30 campioni. Quando il buffer è pieno, la finestra viene convertita in un tensore PyTorch e passata al modello in modalità di inferenza (`torch.no_grad()`). L'errore di ricostruzione, calcolato come MSE tra input e output mediato su tutti i timestep e tutte le feature, viene confrontato con la soglia. Al completamento dello stream, il detector del veicolo viene rimosso e le sue risorse liberate.

### 5.5.3 Smoothing con contatori consecutivi

Per evitare che oscillazioni transitorie dell'errore di ricostruzione generino falsi allarmi, il sistema implementa un meccanismo di smoothing basato su contatori consecutivi. Vengono mantenuti due contatori per ciascun veicolo: un `anomaly_counter`, che si incrementa ad ogni campione con MSE $>$ threshold e si azzera al primo campione normale, e un `normal_counter`, che opera in modo simmetrico. La transizione dallo stato normale allo stato di anomalia avviene solo quando il contatore di anomalie supera la soglia di 20 campioni consecutivi; analogamente, il ritorno allo stato normale richiede 20 campioni consecutivi sotto soglia. Questo meccanismo introduce un ritardo controllato nella segnalazione (20 minuti con campionamento al minuto), ma elimina le notifiche spurie causate da picchi isolati, come quelli associati alle aperture della porta o ai cicli di sbrinamento.

Il risultato dell'analisi viene pubblicato sul topic `fridge/{vehicleName}/anomalies`, includendo l'errore di ricostruzione, lo stato dei contatori e, nelle sole transizioni di stato, un messaggio di allerta che il backend persisterà come notifica.

## 5.6 Gestione delle notifiche

Il backend sottoscrive il topic `fridge/+/anomalies` tramite un channel adapter Spring Integration dedicato. Alla ricezione di un messaggio di anomalia, il service activator verifica la presenza del campo `alertMessage`: se presente, indica una transizione di stato (da normale ad anomalia) e genera una nuova notifica persistente su MongoDB con i campi: nome del veicolo, timestamp, errore di ricostruzione, indice di riga e messaggio di allerta. Un meccanismo di fallback verifica se il messaggio di transizione è stato perso (ad esempio per un'interruzione temporanea della rete): se il flag `anomalyDetected` è attivo ma non esistono notifiche non lette per quel veicolo, ne viene creata una generica.

L'interfaccia del tecnico presenta le notifiche in ordine cronologico con un badge numerico che indica il conteggio delle notifiche non lette. Ciascuna notifica può essere contrassegnata come letta tramite un'azione dedicata, aggiornando lo stato sul database. Da una notifica, il tecnico può accedere direttamente alla dashboard telemetrica del veicolo interessato per analizzare lo storico dei dati e indagare la causa dell'anomalia.

## 5.7 Dashboard telemetrica

La dashboard di monitoraggio consente al tecnico di visualizzare lo storico telemetrico dei veicoli tramite grafici temporali interattivi. L'interfaccia è organizzata in due livelli: un selettore del veicolo, che elenca i veicoli con tratte attive, e un pannello di visualizzazione con grafici ad area realizzati tramite Recharts.

Per ciascun veicolo selezionato, il frontend richiede al backend l'elenco completo dei dati telemetrici memorizzati, ordinati cronologicamente. I dati vengono renderizzati in grafici con codifica cromatica basata sullo stato di guasto: i tratti in condizioni normali appaiono in verde, mentre quelli associati a valori anomali vengono evidenziati in rosso. Il tecnico può filtrare la visualizzazione per singolo parametro (ad esempio, visualizzare solo la temperatura della cella o solo la pressione di scarico) oppure visualizzare tutti i 15 parametri simultaneamente. L'asse temporale mostra il numero progressi-

vo del campione, consentendo di individuare rapidamente il momento in cui il comportamento del sistema è cambiato.

## 5.8 Deployment

L'intero sistema viene eseguito in container Docker orchestrati da Docker Compose. Il file di configurazione definisce i sei servizi, le porte esposte, le variabili d'ambiente e i volumi persistenti per MongoDB. La rete Docker interna consente ai servizi di comunicare tramite i nomi dei container: il backend raggiunge MongoDB tramite l'hostname `mongodb`, il broker MQTT tramite `mosquitto` e il servizio di streaming tramite `fridge-streamer`. L'avvio dell'intero sistema richiede un singolo comando:

```
docker-compose up --build
```

Questa configurazione garantisce la riproducibilità dell'ambiente su qualsiasi macchina dotata del runtime Docker, eliminando le problematiche legate alla configurazione manuale delle dipendenze e delle connessioni di rete tra i servizi.

# Capitolo 6

# Validazione Funzionale

In questo capitolo viene descritta la validazione funzionale della piattaforma ChillChain attraverso uno scenario d'uso completo che attraversa tutte le funzionalità del sistema, coinvolgendo i tre ruoli previsti: amministratore, cliente e tecnico. Lo scenario simula un caso realistico in cui un'azienda di trasporti refrigerati gestisce una flotta di veicoli, riceve richieste di spedizione da più clienti e rileva un'anomalia sull'impianto frigorifero durante il viaggio.

## 6.1 Ambiente di test

La validazione è stata condotta su una macchina locale con l'intero sistema eseguito in container Docker. L'avvio avviene tramite un singolo comando `docker-compose up --build`, che istanzia i sei servizi descritti nel Capitolo 5: il backend Spring Boot, il frontend React, il broker Mosquitto, il servizio di streaming Python, il servizio di anomaly detection e MongoDB. Dopo l'avvio, il frontend è raggiungibile all'indirizzo `http://localhost:5173` e il backend risponde sulla porta 8081.

Per la simulazione della telemetria sono stati predisposti dataset CSV generati dallo script di simulazione termodinamica descritto nella Sezione 5.4: un dataset in condizioni di funzionamento normale per 120 minuti e un dataset con iniezione del guasto `UNDERCHARGE_AND_COND_FOUL` (sottocaricamento del refrigerante combinato a fouling del condensatore) a partire dal minuto 60. Entrambi i dataset sono composti da 120 righe con campionamento al minuto.

## 6.2 Scenario di validazione

Lo scenario è strutturato in cinque fasi che coprono l'intero ciclo di vita di una spedizione, dalla configurazione della flotta fino alla rilevazione di un'anomalia in corso di viaggio.

### 6.2.1 Fase 1 — Configurazione della flotta (Amministratore)

L'amministratore accede alla piattaforma con le proprie credenziali e viene indirizzato alla dashboard di gestione della flotta. Da questa vista, procede alla registrazione di due veicoli refrigerati nella flotta.

*Screenshot: schermata di login*

Figura 6.1: Schermata di login della piattaforma. L'utente inserisce email e password; il sistema restituisce un token JWT e reindirizza alla dashboard corrispondente al ruolo.

Il primo veicolo, denominato `camion1`, è configurato con un vano di carico di $200 \times 200 \times 400$ cm, peso massimo 1000 kg, refrigerazione attiva e costo di 0,50 €/km. Il secondo veicolo, `camion5`, ha dimensioni leggermente inferiori e un costo di 0,40 €/km. Entrambi i veicoli compaiono immediatamente nella lista della flotta.

*Screenshot: form di aggiunta veicolo*

Figura 6.2: Form di registrazione di un nuovo veicolo. L'amministratore specifica il nome identificativo, le dimensioni del vano di carico (larghezza, altezza, lunghezza in cm), il peso massimo trasportabile (kg), la tipologia di refrigerazione e il costo per chilometro (€/km).

### 6.2.2 Fase 2 — Prima spedizione: creazione della tratta (Cliente A)

Un primo cliente accede alla piattaforma e inserisce i dati della propria spedizione: partenza da Milano, arrivo a Roma, pacco di $50 \times 50 \times 50$ cm con peso di 100 kg, refrigerazione necessaria e data di arrivo entro il 28 febbraio 2026.

Il sistema restituisce la lista delle tratte disponibili. In questo caso, poiché nessun veicolo ha ancora una tratta assegnata, vengono mostrati entrambi i veicoli come opzioni "vuote": `camion1` a 287,50 € e `camion5` a 230,00 € (calcolati come distanza Milano–Roma $\times$ prezzo/km del rispettivo veicolo). Per ciascuna opzione, il cliente può visualizzare il percorso su mappa prima di confermare.

Il Cliente A seleziona `camion1`. Il backend crea una nuova entità `Trip` con il percorso Milano–Roma, decrementa la capacità residua del veicolo e salva lo `Shipment` associato.

### 6.2.3 Fase 3 — Seconda spedizione: condivisione della tratta (Cliente B)

Un secondo cliente effettua una ricerca per una spedizione da Bologna a Firenze, con un pacco più piccolo ($30 \times 30 \times 30$ cm, 20 kg, refrigerato, stesso

*Screenshot: lista veicoli della flotta*

Figura 6.3: Lista dei veicoli registrati nella flotta. Per ciascun veicolo sono visualizzati il nome, le dimensioni, il peso massimo, lo stato di refrigerazione e il prezzo per chilometro. Le azioni disponibili includono l'eliminazione del veicolo e la visualizzazione delle tratte associate.

periodo di arrivo).

Il sistema mostra tre opzioni: `camion5` come tratta dedicata (Bologna–Firenze a 46,00 €), `camion1` come tratta condivisa (il percorso ricalcolato Milano–Bologna–Firenze–Roma rientra nella tolleranza di 30 minuti, al prezzo scontato grazie al divisore $2^1$), e un'eventuale ulteriore opzione dedicata. Il Cliente B seleziona la tratta condivisa su `camion1`. Il backend aggiorna la polyline del trip con i nuovi waypoint (Bologna e Firenze come tappe intermedie), decrementa ulteriormente la capacità residua e salva il nuovo `Shipment`.

### 6.2.4 Fase 4 — Gestione della tratta e avvio simulazione (Amministratore)

L'amministratore verifica che la tratta sia stata correttamente aggiornata, visualizzando nella sezione tratte il percorso ricalcolato con i waypoint intermedi e la lista delle spedizioni associate.

L'amministratore avvia quindi la simulazione della telemetria per `camion5`, associato al dataset contenente il guasto combinato. Il backend inoltra la richiesta al servizio di streaming, che inizia a pubblicare i dati sul topic `fridge/camion5/telemetry`.

*Screenshot: form di invio spedizione*

Figura 6.4: Form di ricerca tratte. Il cliente inserisce gli indirizzi di partenza e arrivo, le dimensioni e il peso del pacco, la necessità di refrigerazione e la data di arrivo desiderata.

### 6.2.5 Fase 5 — Rilevazione anomalia e notifica (Tecnico)

Il tecnico accede alla piattaforma con le proprie credenziali e viene indirizzato alla dashboard di monitoraggio. Durante i primi 60 minuti simulati, il modello LSTM Autoencoder ricostruisce correttamente le sequenze: l'errore di ricostruzione oscilla intorno a valori di 0,005–0,008, ben al di sotto della soglia configurata.

A partire dal minuto 60, il dataset inietta il guasto `UNDERCHARGE_AND_COND_FOUL`: il sottocaricamento del refrigerante causa un aumento del surriscaldamento e una riduzione della capacità frigorifera, mentre il fouling del condensatore innalza la temperatura e la pressione di condensazione. Il modello inizia a produrre errori di ricostruzione superiori alla soglia. Dopo 20 campioni consecutivi anomali (il meccanismo di smoothing), il servizio di anomaly detection pubblica un messaggio di allerta sul topic `fridge/camion5/anomalies`.

Il backend riceve il messaggio, crea una notifica persistente e la rende disponibile al tecnico. L'icona delle notifiche nell'header si aggiorna con il badge numerico.

Il tecnico seleziona la notifica e accede alla dashboard telemetrica di `camion5`. I grafici mostrano chiaramente la transizione: intorno al campione 60, la temperatura della cella inizia a salire, il COP diminuisce, la pressione di scarico aumenta e il surriscaldamento cresce sensibilmente.

*Screenshot: lista tratte disponibili con mappa*

Figura 6.5: Risultati della ricerca tratte per il Cliente A. Sono visibili due veicoli disponibili con il percorso Milano–Roma, prezzo calcolato, distanza e durata stimata. La mappa mostra la polyline del percorso selezionato.

## 6.3 Esito della validazione

Lo scenario descritto ha permesso di verificare il corretto funzionamento di tutte le funzionalità della piattaforma lungo l'intero flusso operativo.

Per quanto riguarda il marketplace, la registrazione dei veicoli, la ricerca delle tratte con filtro dimensionale e temporale, il ricalcolo del percorso con waypoint e il pricing con divisore esponenziale hanno prodotto risultati coerenti con le attese: il prezzo della tratta condivisa è risultato effettivamente dimezzato rispetto a quello della tratta dedicata, e la polyline aggiornata ha incluso correttamente le tappe intermedie di Bologna e Firenze.

Sul versante del monitoraggio, il modello LSTM Autoencoder ha ricostruito correttamente le sequenze normali (errore sotto soglia per i primi 60 minuti) e ha rilevato il degrado delle prestazioni a partire dal minuto 60. Il meccanismo di smoothing ha evitato falsi positivi durante la fase normale e ha generato la notifica circa 20 minuti dopo l'inizio del guasto, in linea con il comportamento atteso del contatore a 20 campioni consecutivi. La notifica è stata correttamente persistita su MongoDB e resa disponibile al tecnico, che ha potuto analizzare lo storico dei dati e identificare visivamente la transizione nei grafici telemetrici.

La comunicazione tra i servizi si è dimostrata affidabile: i messaggi MQTT sono stati instradati correttamente dallo streamer all'anomaly detector e da quest'ultimo al backend, senza perdita di dati osservabile. L'avvio e l'arresto della simulazione dal frontend hanno funzionato come previsto, con

*Screenshot: lista tratte con opzione condivisa*

Figura 6.6: Risultati della ricerca tratte per il Cliente B. Oltre al veicolo `camion5` disponibile come tratta dedicata, compare `camion1` come tratta condivisa: il percorso Milano–Roma passa sufficientemente vicino a Bologna e Firenze, la deviazione rientra nei 30 minuti di tolleranza e il prezzo è scontato del 50% grazie alla formula $2^n$ con $n = 1$ spedizione già presente.

il messaggio di completamento che ha attivato la pulizia delle risorse del detector.

La separazione delle viste per ruolo ha operato correttamente: l'amministratore ha avuto accesso esclusivamente alla gestione della flotta e delle tratte, il cliente alla ricerca e prenotazione, il tecnico al monitoraggio e alle notifiche. Tentativi di accesso a endpoint non autorizzati hanno restituito correttamente l'errore HTTP 403.

*Screenshot: dettaglio tratta con spedizioni*

Figura 6.7: Dettaglio della tratta assegnata a `camion1`, con il percorso aggiornato (Milano–Bologna–Firenze–Roma) e la lista delle due spedizioni associate. Sono visibili la distanza totale aggiornata, la durata e la capacità residua del veicolo.

*Screenshot: avvio simulazione telemetrica*

Figura 6.8: Avvio della simulazione telemetrica per il veicolo `camion5`. Il pulsante "Start Simulation" attiva lo streaming dei dati dal CSV al broker MQTT. Lo stato della simulazione mostra il progresso (riga corrente/totale).

*Screenshot: dashboard telemetrica in condizioni normali*

Figura 6.9: Dashboard telemetrica del veicolo `camion5` durante la fase di funzionamento normale (primi 60 minuti). I grafici mostrano i parametri operativi dell'impianto frigorifero: la temperatura della cella ($T_{\text{cab_meas}}$) oscilla intorno al setpoint, il COP è stabile e le pressioni sono nel range atteso.

*Screenshot: notifica di anomalia ricevuta*

Figura 6.10: Pannello notifiche del tecnico. La notifica indica il veicolo interessato (`camion5`), il timestamp dell'allerta, l'errore di ricostruzione e il messaggio "Anomalia rilevata per camion5!". Il badge nell'header mostra il conteggio delle notifiche non lette.

*Screenshot: dashboard telemetrica con anomalia visibile*

Figura 6.11: Dashboard telemetrica del veicolo `camion5` dopo l'iniezione del guasto. A partire dal campione 60, i grafici evidenziano il degrado delle prestazioni: innalzamento di $T_{\text{cab_meas}}$ e $T_{\text{cond_sat}}$, aumento di $P_{\text{dis_bar}}$ e $SH_K$, riduzione del COP. La codifica cromatica passa dal verde al rosso nella zona anomala.

# Capitolo 7

# Conclusioni e Sviluppi Futuri

## 7.1 Conclusioni

L'obiettivo di questo lavoro era la progettazione e l'implementazione di una piattaforma IoT integrata per il trasporto refrigerato condiviso, capace di unire in un unico sistema due funzionalità tradizionalmente separate: un marketplace per il matching tra domanda e offerta di spedizioni e un sistema di monitoraggio della catena del freddo con rilevazione automatica delle anomalie.

Il sistema realizzato, denominato ChillChain, raggiunge questo obiettivo attraverso un'architettura a microservizi che orchestra sei componenti containerizzati: un backend Spring Boot per la logica di business e l'autenticazione, un frontend React differenziato per ruolo, un broker MQTT Mosquitto per la comunicazione asincrona, due servizi Python dedicati rispettivamente allo streaming telemetrico e all'anomaly detection, e un database MongoDB per la persistenza dei dati.

Sul versante commerciale, la piattaforma consente ai clienti di ricercare tratte compatibili con le proprie esigenze di spedizione, visualizzando sia veicoli disponibili sia tratte già programmate su cui è possibile inserirsi. Il meccanismo di validazione geografica, basato sul calcolo della distanza geodetica tra i punti di ritiro/consegna e la polyline del percorso esistente, garantisce che le deviazioni introdotte siano accettabili. Il ricalcolo dinamico del percorso con waypoint intermedi, ordinati tramite un algoritmo greedy con vincoli di precedenza pickup/delivery, aggiorna automaticamente il tracciato della tratta. Il sistema di pricing con divisore esponenziale $2^n$ crea un incentivo economico crescente alla condivisione, rendendo il trasporto progressivamente più conveniente al crescere dell'occupazione del veicolo.

Sul versante del monitoraggio, il modello LSTM Autoencoder addestrato

su dati di funzionamento normale è in grado di rilevare anomalie complesse e multivariate che sfuggirebbero a sistemi basati su soglie statiche, come il degrado graduale delle prestazioni del compressore o il fouling del condensatore combinato a sottocariamento del refrigerante. Il meccanismo di smoothing a contatori consecutivi riduce efficacemente i falsi positivi, introducendo un ritardo controllato nella segnalazione a fronte di una maggiore affidabilità delle notifiche. L'intero flusso — dalla generazione della telemetria alla notifica persistente per il tecnico — opera in modo automatico e trasparente, senza richiedere intervento manuale.

La validazione funzionale, condotta attraverso uno scenario end-to-end che ha coinvolto tutti e tre i ruoli previsti, ha confermato il corretto funzionamento dell'intera catena: registrazione dei veicoli, ricerca e prenotazione delle tratte con pricing dinamico, avvio della simulazione, rilevazione dell'anomalia dopo il periodo di smoothing e visualizzazione dello storico telemetrico da parte del tecnico.

## 7.2 Sviluppi futuri

Nonostante i risultati ottenuti, il lavoro presenta margini di miglioramento che possono essere affrontati in sviluppi successivi.

**Ottimizzazione temporale dei waypoint.** L'attuale sistema di matching tra spedizioni e tratte esistenti si concentra prevalentemente sulla compatibilità spaziale: la distanza dei punti di ritiro e consegna dalla polyline del percorso e la tolleranza sulla durata complessiva del viaggio. Non viene tuttavia stimato l'orario di passaggio del veicolo nei punti intermedi. In uno scenario reale, un cliente che richiede il ritiro a Bologna alle 10:00 non ha garanzia che il veicolo in transito da Milano a Roma passi effettivamente in quella fascia oraria. Un'evoluzione del sistema potrebbe calcolare l'ETA (*Estimated Time of Arrival*) in ciascun waypoint sulla base della velocità media e della posizione lungo il percorso, consentendo al sistema di rifiutare combinazioni temporalmente incompatibili e al cliente di conoscere l'orario previsto di ritiro e consegna.

**Retraining periodico del modello di anomaly detection.** Il modello LSTM Autoencoder attualmente impiegato è stato addestrato offline su un dataset sintetico e caricato come artefatto statico nel servizio di inferenza. In un contesto operativo reale, le condizioni di funzionamento degli impianti frigoriferi possono variare nel tempo a causa dell'usura, della manutenzione o dell'introduzione di nuovi veicoli con caratteristiche diverse. Un sistema

di raccolta continua dei dati telemetrici in produzione, combinato con una pipeline di retraining periodico, consentirebbe di aggiornare il modello e la soglia di anomalia per riflettere l'evoluzione della flotta, migliorando la sensibilità del rilevamento e riducendo i falsi positivi su veicoli con profili operativi diversi da quelli presenti nel dataset di addestramento.

**Supporto allo streaming multi-veicolo simultaneo.** Il servizio di anomaly detection è già progettato per gestire più veicoli contemporaneamente, creando dinamicamente un'istanza di detector per ciascun veicolo alla ricezione del primo messaggio. Il servizio di streaming, tuttavia, supporta un solo stream alla volta: l'avvio di una seconda simulazione richiede l'arresto della prima. L'estensione a stream paralleli, ciascuno associato a un dataset e a un thread indipendente, consentirebbe di simulare scenari più realistici con più veicoli in viaggio simultaneamente, sollecitando il sistema di anomaly detection nella gestione concorrente di buffer e contatori multipli.

**Notifiche push in tempo reale.** Nel sistema attuale, il tecnico visualizza le notifiche interrogando periodicamente il backend tramite richieste REST. L'integrazione di un canale di notifiche push — ad esempio tramite WebSocket o *Server-Sent Events* — consentirebbe di recapitare le allerte al tecnico nell'istante stesso in cui vengono generate, senza necessità di ricaricare la pagina o di implementare meccanismi di polling. Questo miglioramento risulterebbe particolarmente rilevante in contesti operativi in cui la tempestività dell'intervento è critica per la salvaguardia della merce trasportata.

# Bibliografia


[1] Bilge Atasoy, Frederik Schulte e Alex Steenkamp. «Platform-Based Collaborative Routing using Dynamic Prices as Incentives». In: *Transportation Research Record* 2674.10 (2020), pp. 670–684. DOI: 10.1177/0361198120935116.

[2] Flock Freight. *FlockDirect® Shared Truckload*. 2024. URL: https://www.flockfreight.com (visitato il 24/02/2025).

[3] James Gillespie et al. «Real-Time Anomaly Detection in Cold Chain Transportation Using IoT Technology». In: *Sustainability* 15.3 (2023), p. 2255. DOI: 10.3390/su15032255.

[4] Quicargo B.V. *Digital Freight Platform for Road Transport and Logistics*. 2024. URL: https://quicargo.com (visitato il 24/02/2025).

[5] Mikhail Samoilov. *Simulated Refrigerator Fault Diagnosis Dataset*. 2024. URL: https://www.kaggle.com/datasets/samoilovmikhail/simulated-refrigerator-fault-diagnosis-dataset (visitato il 24/02/2025).

[6] Phuong Hanh Tran, Cédric Heuchenne e Sébastien Thomassey. «Forecasting and Anomaly Detection approaches using LSTM and LSTM Autoencoder techniques with the applications in supply chain management». In: *International Journal of Information Management* 57 (2021), p. 102282. DOI: 10.1016/j.ijinfomgt.2020.102282.